

Lazy Evaluation, Future & Nebenläufigkeit

Lazy Evaluation, Future & Nebenläufigkeit

Gliederung



- 1. Lazy Evaluation & LazyList
- 2. Future
- 3. ExecutionContext
- 4. Nebenläufigkeit & Parallelität
- 5. Übung

Lazy Evaluation & LazyList

Lazy Evaluation & LazyList

Motivation: Nicht alles sofort berechnen



- Manche Berechnungen sind **teuer**. Andere brauchen wir am Ende gar nicht.
- Zwei wichtige Fragen:
 - **Wann** wird ein Ausdruck ausgewertet?
 - **Wie oft** wird er ausgewertet?
- Scala bietet dafür mehrere Werkzeuge:
 - ``val``
 - ``def``
 - ``lazy val``
 - ``LazyList``

Lazy Evaluation & LazyList

`val`, `def` und `lazy val`

- `val`: wird **sofort** ausgewertet und gespeichert
- `def`: wird bei **jedem Aufruf neu** ausgewertet
- `lazy val`: wird **beim ersten Zugriff** ausgewertet und danach gespeichert

```
def expensive(label: String): Int = {  
    println(s"berechne $label")  
    42  
}
```

```
val eager: Int = expensive("val")  
def repeated: Int = expensive("def")  
lazy val delayed: Int = expensive("lazy val")
```

Lazy Evaluation & LazyList

Beobachtung: Wann passiert die Auswertung?

- ``val`` läuft schon bei der Definition
- ``def`` läuft bei jedem Zugriff erneut
- ``lazy val`` läuft genau **einmal**, aber erst bei Bedarf

```
println("Start")
```

```
val eager: Int = expensive("val")
```

```
def repeated: Int = expensive("def")
```

```
lazy val delayed: Int = expensive("lazy val")
```

```
println("---")
```

```
repeated
```

```
repeated
```

```
delayed
```

```
delayed
```

Lazy Evaluation & LazyList

Call by Value

- Standardfall in Scala: Argumente werden **vor dem Aufruf** ausgewertet
- Der Funktionskörper bekommt dann bereits das berechnete Ergebnis
- ``next()`` wird **vor** ``twice`` genau einmal ausgewertet

```
def twice(x: Int): Int = x + x
```

```
def next(): Int = {  
    println("berechne next()")  
    21  
}
```

```
twice(next())
```

Lazy Evaluation & LazyList

Call by Name

- Mit ``=> A`` wird das Argument **nicht sofort** ausgewertet
- Statt eines Werts wird der Ausdruck verzögert an die Funktion übergeben
- Jetzt kann ``next()`` im Funktionskörper **mehrfach** ausgewertet werden

```
def twice(x: => Int): Int = x + x
```

```
def next(): Int = {  
    println("berechne next()")  
    21  
}
```

```
twice(next())
```


Lazy Evaluation & LazyList

Call by Value vs. Call by Name

- ``x: Int` = Wert` wird übergeben
- ``x: => Int` = Ausdruck` wird übergeben
- Call by Name ist praktisch, wenn die Funktion den Ausdruck vielleicht gar nicht braucht
- ``expensive(...)`` wird hier gar nicht ausgewertet

```
def choose(flag: Boolean, value: => Int): Int = {  
    if (flag) value else 0  
}
```

```
choose(false, expensive("unnötig"))
```

- Ein by-name-Parameter wird bei jedem Zugriff erneut berechnet
- Wenn wir das Ergebnis nur einmal brauchen, können wir lokal ein `lazy val` daraus machen
- Die Idee ist einfach: spät starten, einmal berechnen, dann wiederverwenden

```
def useTwice(x: => Int): Int = {  
    lazy val cached: Int = x  
    cached + cached  
}
```

```
useTwice(expensive("by-name"))
```

Lazy Evaluation & LazyList

Wann ist `lazy val` sinnvoll?



- Für **teure Initialisierung**, die vielleicht nie gebraucht wird
- Für Werte, die **später**, aber dann nur **einmal** berechnet werden sollen
- Für rekursive oder große Strukturen, die stückweise genutzt werden
- Für kleine Werte lohnt sich `lazy val` oft nicht. Es macht die Auswertung und Synchronisation aufwendiger.

```
lazy val config: Map[String, String] = loadConfig()
```

`LazyList`: Werte bei Bedarf erzeugen

- `LazyList[A]` ist eine Folge, deren Elemente **erst bei Bedarf** berechnet werden
- Das ist praktisch für große oder sogar unendliche Sequenzen
- Passt gut zu `map`, `filter`, `take`

```
val naturals: LazyList[Int] = LazyList.from(1)
```

Lazy Evaluation & LazyList

Unendliche Sequenzen sind möglich

- `naturals` ist konzeptionell unendlich
- Trotzdem funktioniert `take(5)`, weil nur die ersten Elemente berechnet werden

```
val naturals: LazyList[Int] = LazyList.from(1)
```

```
val firstFive: List[Int] =  
  naturals.take(5).toList
```

Lazy Evaluation & LazyList

`LazyList` mit Transformationen

- Auch `map` und `filter` bleiben zunächst **lazy**
- Erst `take(...).toList` erzwingt konkrete Werte

```
val evens: LazyList[Int] =  
    LazyList.from(1)  
        .map(_ * 2)
```

```
val result: List[Int] =  
    evens.filter(_ % 4 == 0)  
        .take(5)  
        .toList
```

Lazy Evaluation & LazyList

Merksatz: lazy ist nicht async



- **Lazy** bedeutet: später auswerten
- **Async** bedeutet: Berechnung läuft getrennt von der aktuellen Ausführung weiter
- ``LazyList`` und ``lazy val`` verzögern Berechnung
- ``Future`` startet meistens **sofort**. Das Ergebnis kommt aber erst später.

Future

- Viele reale Programme warten auf:
 - Netzwerk
 - Datenbank
 - Dateisystem
 - externe Services
- Währenddessen soll das Programm oft **weitere Arbeit** erledigen können
- `Future[A]` beschreibt eine asynchrone Berechnung, die später **ein Ergebnis oder einen Fehler** liefert

Future

Erstes `Future`

- Der Wert ist nicht sofort da
- Statt `Int` bekommen wir zunächst ein `Future[Int]`

```
import scala.concurrent.ExecutionContext
import scala.concurrent.Future

given ExecutionContext = ExecutionContext.global

val f: Future[Int] = Future {
  // simuliert eine langsamere Berechnung
  42
}
```

ExecutionContext

Was ist ein `ExecutionContext`?

- Ein `ExecutionContext` beschreibt, **wo und wie** asynchrone Aufgaben ausgeführt werden
- Einfach gesagt verteilt er Arbeit auf Threads
- `Future` braucht ihn, um seine Berechnung starten zu können

```
import scala.concurrent.ExecutionContext
```

ExecutionContext

`given` und `using` in der Praxis

- Genau wie in VL04: `using` fordert Kontext an, `given` stellt ihn bereit

```
import scala.concurrent.ExecutionContext
import scala.concurrent.Future
```

```
def loadValue()(using ec: ExecutionContext): Future[Int] =
  Future { 42 }
```

```
given ExecutionContext = ExecutionContext.global
```

```
val f: Future[Int] = loadValue()
```

ExecutionContext

`ExecutionContext.global`



- Der globale Context ist für viele Beispiele bequem
- Intern basiert er auf einem gemeinsam genutzten Thread Pool
- Für Lehrbeispiele ist das meist ausreichend
- In größeren Anwendungen wählt man den Context meist gezielter

```
given ExecutionContext = ExecutionContext.global
```

Warum nicht einfach immer `global`?

- `ExecutionContext.global` wird von vielen Aufgaben gemeinsam genutzt
- Wenn dort blockierende oder sehr lange Arbeit landet, kann das andere Aufgaben ausbremsen
- Dann konkurrieren fachlich völlig verschiedene Jobs um denselben Pool
- Für kleine Demos ist `global` okay. In echten Systemen ist es oft nur ein Startpunkt.

- Ein Thread Pool ist eine verwaltete Menge von Worker-Threads
- Neue Aufgaben erzeugen dann nicht jedes Mal einen neuen Thread
- Stattdessen landen sie in einer Warteschlange und freie Threads arbeiten sie nacheinander ab
- Das spart Overhead und begrenzt gleichzeitig die Parallelität

ExecutionContext

Eigenen Thread Pool definieren

- Für bestimmte Aufgaben kann man bewusst einen eigenen Pool verwenden
- Daraus lässt sich dann ein eigener `ExecutionContext` bauen
- Die Wahl des Contexts ist also mehr als ein Syntaxdetail. Sie beeinflusst das Laufzeitverhalten.

```
import java.util.concurrent.Executors
import scala.concurrent.ExecutionContext

val executor = Executors.newFixedThreadPool(4)
given ExecutionContext = ExecutionContext.fromExecutor(executor)
```

Problem: Blockierender Code

- `Future` ist besonders sinnvoll für **nicht-blockierende** Arbeit
- Wenn ein Thread lange blockiert, fehlt er für andere Aufgaben
- `Thread.sleep`, blockierendes I/O oder lange Locks können Thread Pools ausbremsen

```
val f: Future[Int] = Future {  
    Thread.sleep(5000)  
    42  
}
```

- Nicht unbedacht blockierende Operationen in `ExecutionContext.global` legen
- CPU-lastige und I/O-lastige Arbeit gedanklich trennen
- Kontext explizit durchreichen, wenn er fachlich wichtig ist
- `Future`-Code klein halten und Seiteneffekte begrenzen

Future

Future

`Future` ist eager

- `Future { ... }` startet in der Regel **sofort**
- Das ist ein zentraler Unterschied zu `lazy val` oder `LazyList`
- Die Berechnung startet also nicht erst beim ersten Zugriff

```
val f: Future[Int] = Future {  
    println("Future startet")  
    42  
}
```

```
println("nach der Definition")
```

- Wie schon bei `Option`, `Either` oder `List` können wir Ergebnisse transformieren
- Genau wie `flatMap` braucht auch `map` auf `Future` einen `ExecutionContext`
- Das Ergebnis bleibt asynchron. Aus `Future[Int]` wird wieder `Future[Int]`.

```
val f: Future[Int] = Future { 21 }
```

```
val doubled: Future[Int] =  
  f.map(x => x * 2)
```

- `flatMap` verkettet abhängige asynchrone Schritte
- Auch `flatMap` verwendet einen `ExecutionContext` für die Weiterverarbeitung
- Deshalb funktionieren auch `for`-Comprehensions mit `Future`
- Das Muster kennen wir also schon: `map`, `flatMap`, `for`

```
def loadUser(id: Int): Future[String] = Future { s"user-$id" }  
def loadScore(user: String): Future[Int] = Future { user.length }
```

```
val program: Future[Int] = for {  
  user  <- loadUser(1)  
  score <- loadScore(user)  
} yield score
```

Future

Sequenziell komponiert

- Der zweite Schritt hängt vom ersten ab
- Diese Verkettung ist **logisch sequenziell**

```
def loadProfile(): Future[String] = Future { "profil" }  
def loadOrders(profile: String): Future[Int] = Future { profile.length }  
  
val result: Future[Int] = for {  
  profile <- loadProfile()  
  orders  <- loadOrders(profile)  
} yield orders
```


Future

Nebenläufig vorbereitet

- Beide Futures werden **vorher** gestartet
- Danach kombinieren wir nur noch ihre Ergebnisse

```
def loadA(): Future[Int] = Future { 10 }
```

```
def loadB(): Future[Int] = Future { 32 }
```

```
val fa: Future[Int] = loadA()
```

```
val fb: Future[Int] = loadB()
```

```
val combined: Future[Int] = for {
```

```
  a <- fa
```

```
  b <- fb
```

```
} yield a + b
```

- `Future.sequence` dreht `List[Future[A]]` in `Future[List[A]]`
- `Future.traverse` kombiniert `map` und `sequence`
- `sequence` und `traverse` kombinieren Futures. Echte Parallelität garantieren sie nicht.

```
val futures: List[Future[Int]] = List(loadA(), loadB())
```

```
val all: Future[List[Int]] = Future.sequence(futures)
```

```
val traversed: Future[List[Int]] =  
  Future.traverse(List(1, 2, 3))(id => Future { id * 10 })
```

Future

Ergebnisse beobachten

- Ein `Future` liefert sein Ergebnis **später**
- Deshalb reagieren wir oft mit Callback-Methoden auf Erfolg oder Fehler
- Im Anwendungscode arbeiten wir meist lieber mit `map` und `flatMap`
- `Await` ist eher ein Hilfsmittel für REPL, kleine Demos oder Tests

```
import scala.util.Failure
import scala.util.Success
```

```
f.onComplete {
  case Success(value) => println(s"Erfolg: $value")
  case Failure(error) => println(s"Fehler: ${error.getMessage}")
}
```

Future

Fehler in `Future`

- Ein `Future` kann erfolgreich sein oder mit einer Exception scheitern
- Fehler werden ebenfalls asynchron transportiert
- Fachliche Fehler und technische Fehler sind dabei zunächst nicht getrennt

```
val f: Future[Int] = Future {  
    throw new RuntimeException("boom")  
}
```

Future

Fehler behandeln

- ``recover``: aus Fehler wird ein Ersatzwert
- ``recoverWith``: aus Fehler wird ein neues ``Future``

```
val safe: Future[Int] =  
  f.recover {  
    case _: RuntimeException => 0  
  }
```

```
val retried: Future[Int] =  
  f.recoverWith {  
    case _: RuntimeException => Future { 1 }  
  }
```

`Future[Either[E, A]]` für fachliche Fehler

- `Future` steht für **Asynchronität**
- `Either[E, A]` steht für **fachlichen Erfolg oder Fehler**
- Beides lässt sich kombinieren
- So lässt sich der Typ lesen: Die Berechnung läuft asynchron und liefert später entweder einen Fehler oder ein Ergebnis.

```
type AppResult[A] = Future[Either[String, A]]
```

Nebenläufigkeit & Parallelität

Nebenläufigkeit & Parallelität

Nebenläufigkeit



- Mehrere Aufgaben sind in einem Zeitraum **gleichzeitig in Arbeit**
- Die Ausführung kann verschachtelt oder abwechselnd stattfinden
- Es geht um **Struktur und Koordination**
- Beispiel:
 - Anfrage A wartet auf Netzwerk
 - in der Zeit bearbeitet das System Anfrage B

Nebenläufigkeit & Parallelität

Parallelität



- Mehrere Aufgaben laufen **tatsächlich gleichzeitig**
- Das braucht in der Regel mehrere Kerne oder Hardware-Ressourcen
- Es geht um **reale Gleichzeitigkeit**
- Beispiel:
 - Kern 1 berechnet A
 - Kern 2 berechnet B

Nebenläufigkeit & Parallelität

Wichtig: `Future` bedeutet nicht automatisch Parallelität



- `Future` beschreibt asynchrone Ausführung
- Ob Aufgaben wirklich parallel laufen, hängt von Laufzeit und Thread Pool ab
- Auch viele Futures können am Ende nur abwechselnd abgearbeitet werden

Nebenläufigkeit & Parallelität

Drei Sichtweisen

- **Sequenziell:** Aufgabe B startet erst nach Aufgabe A
- **Nebenläufig:** Beide Aufgaben sind in Arbeit, wechseln sich aber ggf. ab
- **Parallel:** Beide Aufgaben laufen gleichzeitig auf unterschiedlichen Ressourcen
- Dieser Code startet zwei Futures, garantiert aber noch keine echte Parallelität

```
val a = loadA()
```

```
val b = loadB()
```

Nebenläufigkeit & Parallelität

Typischer Denkfehler bei `for`

- Eine `for`-Comprehension beschreibt zuerst die **Datenabhängigkeiten**
- Sie sagt nicht automatisch: "Bitte parallel ausführen"
- Hier wird `loadB()` erst aufgerufen, wenn `a` da ist

```
for {  
  a <- loadA()  
  b <- loadB()  
} yield a + b
```

Nebenläufigkeit & Parallelität

Futures erst starten, dann kombinieren



- Als Faustregel hilft oft:
 - unabhängige Aufgaben früh starten
 - abhängige Aufgaben mit `flatMap` oder `for` verketteten

```
val fa = loadA()
```

```
val fb = loadB()
```

```
for {  
  a <- fa  
  b <- fb  
} yield a + b
```

Nebenläufigkeit & Parallelität

Zusammenfassung: Laufzeitverhalten richtig lesen



- ``lazy val`` und ``LazyList``: **spätere** Auswertung
- ``Future``: **asynchrone** Ausführung
- ``ExecutionContext``: Ausführungsumgebung
- Nebenläufigkeit und Parallelität sind **nicht dasselbe**
- Startzeitpunkt und Abhängigkeiten bestimmen das Verhalten

Übung

- Lege eine kleine Beispieldomäne an
- Mit ihr arbeiten wir in allen weiteren Teilaufgaben weiter

```
case class Coordinates(latitude: Double, longitude: Double)
case class City(name: String, coordinates: Coordinates)
```


- Schreibe eine kleine Hilfsfunktion mit `println`, die eine Berechnung sichtbar macht
- Lege denselben Ausdruck als `val`, `def` und `lazy val` ab
- Tipp: achte darauf, wann und wie oft jeweils ausgewertet wird

- Implementiere zwei Funktionen:
 - eine mit Parameter ``x: Int``
 - eine mit Parameter ``x: => Int``
- Verwende einen Ausdruck mit Seiteneffekt, um den Unterschied sichtbar zu machen
- Tipp: ein by-name-Parameter wird bei jedem Zugriff erneut ausgewertet

- Schreibe eine Funktion, die einen by-name-Parameter intern in ein `lazy val` legt
- Tipp: so kannst du verzögert starten und den Wert trotzdem nur einmal berechnen

- Erzeuge eine `LazyList[Int]` ab `1000`
- Filtere zuerst alle geraden Zahlen heraus
- Quadriere diese Zahlen und nimm dann die ersten `8` Ergebnisse
- Gib das Resultat als `List[Int]` zurück

- Schreibe eine kleine asynchrone Funktion mit `(using ExecutionContext)`
- Stelle den Context über ein `given` bereit
- Tipp: orientiere dich an VL04 und an den `given`/`using`-Folien

```
import scala.concurrent.ExecutionContext
import scala.concurrent.Future
```

- Implementiere eine Funktion `loadTemperature(city: City): Future[Double]`
- Tipp: simuliere nur eine langsame Berechnung. Die eigentliche Logik ist hier noch nicht wichtig.
- Ziel: ein erster asynchroner Schritt für die spätere Pipeline

Teil 8: ``map``, ``flatMap`` und ``for`` wiederholen

- Nutze ab jetzt das bereitgestellte Übungsprojekt mit ``WeatherService`` aus Github zur aktuellen Vorlesung
- Kombiniere zwei asynchrone Service-Aufrufe
- Tipp: schreibe die Verkettung einmal mit ``flatMap`` und einmal mit ``for``

- Handle einen Fehlerfall aus dem bereitgestellten `WeatherService`
- Fange den Fehler mit `recover` oder `recoverWith` ab
- Tipp: gib für den Fehlerfall einen sinnvollen Default-Wert oder ein Ersatz-`Future` zurück

Teil 10: Sequenziell oder nebenläufig?

- Betrachte zuerst diese erste Variante:
- Erkläre in eigenen Worten den Unterschied beim Startzeitpunkt
- Tipp: entscheidend ist nicht nur das `for`, sondern **wann** `loadA()` und `loadB()` aufgerufen werden

```
for {  
  a <- loadA()  
  b <- loadB()  
} yield a + b
```

Teil 10: Sequenziell oder nebenläufig? (2)

- Vergleiche sie jetzt mit dieser zweiten Variante:
- Zusatzfrage: Welche Variante ist näher an echter Nebenläufigkeit?

```
val fa = loadA()
```

```
val fb = loadB()
```

- Baue jetzt aus mehreren Bausteinen eine gemeinsame Aufgabe:
 - nutze den bereitgestellten `WeatherService` aus dem Projekt
 - starte mit einer kleinen `List[City]`
 - rufe Wetterdaten asynchron über `Future` ab
 - verarbeite zwei oder mehr API-Aufrufe in einem `for`-`yield``
- Zusatzfrage: Welche Teile sind **asynchron**, welche nur **normal ausgewertet**?

```
// Mögliche Bausteine:  
// val cities = List(  
//   City("München", Coordinates(48.137154, 11.576124)),  
//   City("Berlin", Coordinates(52.520008, 13.404954))  
// )  
//  
// val service = ...  
//  
// for {  
//   munich <- ...  
//   berlin <- ...  
// } yield ...
```